

Tanglish

A beginner-friendly programming language with phonetic Tamil keywords in Roman script - transpiled to JavaScript

R S Surjune (25BDS057) | Shri Hari A K (25BDS050) | Guide: Dr. B. Radha
Department of Computer Technology and Data Science, Sri Krishna Arts and Science College

1. What is Tanglish?

When a Tamil-medium student learns programming, they must learn two things at once: coding logic AND English vocabulary. Words like **function**, **return**, **if** and **else** become barriers before the logic is even reached. Tanglish removes that barrier: every keyword is a phonetic Tamil word written in Roman script - **enil** instead of **if**, **thiruppi** instead of **return** - so the student's full attention goes to the logic.

Programs are written in files ending with **.tml** and run with one command, exactly like Python:

```
tanglish run marks_report.tml
```

2. How it works - the pipeline

Tanglish is a **transpiler**, not a compiler or virtual machine. It translates .tml source into readable JavaScript and lets Node.js (a mature, battle-tested engine) do the running - the same approach TypeScript uses. Four stages:



One program traced through all four stages

Take this two-line program:

SOURCE (program.tml)

```
mark = 45
achchu(mark + 5)
```

Stage 1 - Lexer (hand-written, character by character). It recognises words and symbols, not grammar. Every token records its line and column - that is how every later error can point to the exact place:

TOKENS

```
IDENTIFIER "mark"  ASSIGN "="  NUMBER "45"  NEWLINE
PRINT "achchu"  LPAREN "("  IDENTIFIER "mark"  PLUS "+"
NUMBER "5"  RPAREN ")"  NEWLINE  EOF
```

Stage 2 - Parser (built with the Chevrotain library) checks the token order against the grammar and builds the **AST** (Abstract Syntax Tree) - a tree of plain objects that captures structure, not spelling:

AST

```
Program
|- Assignment name:"mark"
|   |- NumberLiteral 45
|   |- PrintStatement
|       |- BinaryExpression "+"
|           |- Identifier "mark"
|           |- NumberLiteral 5
```

Stage 3 - Transpiler walks the tree and prints JavaScript. It remembers that `mark` is a new name (so it gets `let`) and knows `achchu` means `console.log`:

GENERATED JAVASCRIPT

```
let mark = 45;
console.log(mark + 5);
```

Stage 4 - Node.js executes that text. The terminal prints **50**.

3. Code architecture - what each file does

File	Job
<code>src/keywords.js</code>	The dictionary: 18 Tanglish keywords mapped to token types. Case-insensitive lookup. The ONLY place the vocabulary lives.
<code>src/lexer.js</code>	Hand-written scanner: characters in, tokens out. Rejects floats, unknown symbols and unclosed strings with line-numbered errors.
<code>src/errors.js</code>	Three error classes sharing one parent (<code>TanglishError</code>), so the CLI catches every language error with a single check.
<code>src/tokens.js</code>	Bridge to Chevrotain: declares its token vocabulary and adapts our tokens into the shape it expects. Chevrotain's own lexer is deliberately bypassed - our hand-written lexer is the scanner.
<code>src/parser.js</code>	The grammar (about 20 rules) plus the <code>AstBuilder</code> visitor that turns Chevrotain's raw tree into the clean AST.
<code>src/builtins.js</code>	The 10 built-in functions (<code>ullidu</code> , <code>neelam</code> , ...) as ready-made JavaScript snippets - functions, not keywords.
<code>src/transpiler.js</code>	Walks the AST, emits indented JavaScript: <code>let</code> vs reassignment, <code>marathu</code> constants, Python-style hoisting, precedence-aware parentheses; pastes in only the built-ins actually used.
<code>cli.js</code>	The command line (<code>Commander.js</code>): <code>tanglish run file.tml [--show-js]</code> . Friendly errors, Windows BOM handling, executes the generated JS.

Tests live in `test/` - one suite per stage, about 100 checks in total, run with `npm test`. The transpiler suite does not just inspect the generated JavaScript as text: it executes it and verifies the printed output. Formal grammar and AST reference: `docs/GRAMMAR.md`; guided tour: `docs/ARCHITECTURE.md`.

4. The language - syntax reference

Keywords are **case-insensitive** (`enil` = `Enil` = `ENIL`); variable names stay case-sensitive. Statements end at the end of a line (semicolons optional), comments run from `//` to the end of the line, and source numbers are whole numbers only.

Keywords (18)

Tanglish	Meaning	JavaScript
seyal	define a function	function
enil	if	if
illaenil	else-if	else if
illana	else	else
thiruppi	return	return
achchu	print	console.log
unmei / poi	true / false	true / false
onnumilai	null (nothing)	null
varai	while loop	while
mindum	for loop	for
ulla	in - joins mindum to a range or list	of
niruthu	break out of a loop	break
thodar	skip to the next round	continue
matrum	and	&&
allathu	or	
alla	not	!
marathu	constant (assigned once)	const

Built-in functions (10) - ready-made functions, not keywords

Tanglish	Does	Example
ullidu	read what the user types	peyar = ullidu("Name: ")
neelam	length of a string or list	neelam(marks)
innaipu	add to the end of a list	innaipu(marks, 77)
neekku	remove and return the last item	neekku(marks)
enn	convert to a number	enn("5") + 1 -> 6
urai	convert to text	urai(5) + "0" -> "50"
ehtho	random number between 0 and 1	ehtho()
muulu	round to nearest whole number	muulu(7 / 2) -> 4
uchcham	biggest of the given values	uchcham(10, 25) -> 25
vagai	kind of a value	vagai([1]) -> "list"

Variables, lists and operators

Variables are Python-style - just assign: `pass_marku = 50` (the transpiler emits `let` on first assignment).
Lists: `marks = [80, 65, 92]`, first item `marks[0]`, change a slot with `marks[0] = 99`. Arithmetic and comparison operators: `+` `-` `*` `/` `>` `<` `>=` `<=` `==` `!=` and the range symbol `...`. A variable born inside a branch or loop survives after it, like Python.

Operator precedence (weakest to strongest)

```
allathu(or) < matrum(and) < alla(not) < comparisons  
< + - < * / < unary minus < literals, calls, ( ), [i]
```

so: $2 + 3 * 4$ is 14 (multiplication binds tighter)
a matrum b allathu c groups as (a and b) or c

5. Examples

5.1 Grade check - functions, if/else, string joining

examples/grade_check.tml

```
pass_marku = 50  
seyal match_check(peyar, score) {  
  enil (score >= pass_marku) {  
    thiruppi peyar + "pass"  
  }  
  illana {  
    thiruppi peyar + "fail"  
  }  
}  
achchu(match_check("Surjune", 87))
```

OUTPUT

Surjunepass

Run it with `--show-js` to see the translation: `seyal` became `function`, `enil/illana` became `if/else`, `achchu` became `console.log`, and the first assignment gained a `let`:

GENERATED JAVASCRIPT

```
let pass_marku = 50;  
function match_check(peyar, score) {  
  if (score >= pass_marku) {  
    return peyar + "pass";  
  } else {  
    return peyar + "fail";  
  }  
}  
console.log(match_check("Surjune", 87));
```

5.2 Loops, lists and grading - the v2 features together

examples/marks_report.tml (essential part)

```
marathu THERCHI = 50
seyal grade(m) {
  enil (m >= 90)      { thiruppi "A" }
  illaenil (m >= 75)  { thiruppi "B" }
  illaenil (m >= THERCHI) { thiruppi "C" }
  illana              { thiruppi "F" }
}

marks = [92, 67, 45, 88]
innaipu(marks, 71)      // list grows to 5 items

total = 0
mindum m ulla marks {   // walk every item
  total = total + m
}
achchu("Average: " + urai(muulu(total / neelam(marks))))

mindum i ulla 1 .. neelam(marks) { // count 1 to 5
  achchu(urai(marks[i - 1]) + " - " + grade(marks[i - 1]))
}
```

OUTPUT

```
Average: 73
92 - A
67 - C
45 - F
88 - B
71 - C
```

5.3 Interactive program - keyboard input

examples/number_game.tml (essential part)

```
secret = muulu(ehtho() * 9) + 1 // random 1..10
varai (unmei) {                // loop forever
  guess = enn(ullidu("Un guess enna? "))
  enil (guess == secret) {
    achchu("Correct! Nee jeichitte!")
    niruthu // break
  }
  illaenil (guess < secret) {
    achchu("Innum konjam periya number...")
  }
  illana {
    achchu("Innum konjam chinna number...")
  }
}
```

6. Friendly errors - never a stack trace

Every stage reports mistakes as one plain sentence with the line number. The student never sees JavaScript internals:

```
mark = 98.5
  Lexer error on line 1: float literals like '98.5...' are not
  supported yet - Tanglish currently accepts whole numbers only.

enil (x > 1 {
  Parser error on line 1: expected ')' but found '{'.

marathu P = 5   then   P = 6
  Transpiler error on line 2: 'P' is a marathu constant - its
  value can never be changed.

achchu(illatha_seyal(5))
  Runtime error: illatha_seyal is not defined
```

7. Running and testing

Command	What it does
<code>tanglish run file.tml</code>	transpile and run a program (works from any folder after npm link)
<code>tanglish run file.tml --show-js</code>	also print the generated JavaScript
<code>node cli.js run file.tml</code>	same, without the global command
<code>npm test</code>	run all three test suites (about 100 checks)

Requirements: Node.js 20.19 or newer. Setup: `npm install` once, then `npm link` to register the global tanglish command. Source code: github.com/Surjune/Hajune.